

Contents

ÖZET	2
1 GİRİŞ.....	2
2 PROJENİN AMACI VE KAPSAMI	3
3 KULLANILAN DONANIM VE YAZILIMLAR.....	4
3.1 DONANIM TABLOSU	4
3.2 YAZALIM TABLOSU	4
4 SİSTEM MİMARISI.....	5
5 DEVRE BAĞLANTISI VE DEVRE ŞEMASI	6
5.1 DEVRE BAĞLANTISI TABLOSU.....	6
5.2 DEVRE ŞEMASI.....	6
6 VERİ TOPLAMA SÜRECİ	7
7 EDGE IMPULSE MODEL EĞİTİMİ	8
7.1 MODEL SONUÇLARI	9
8 BU PROJEDE NEDEN TINYML MODELİ KULLANILDI?	10
9 TINYML'IN ESP32'YE BAĞLANMASI	10
10 KARŞILAŞILAN PROBLEMLER VE ÇÖZÜMLER.....	11
11 SONUÇ VE DEĞERLENDİRME	12
KAYNAKÇA.....	12

Özet

Bu projede, bir gömülü sistem üzerinde TinyML kullanılarak motor rulmanı arızası tespiti için yenilikçi bir yaklaşım sunan “IoT device for detecting abnormal vibrations in motors using TinyML” makalesi referans alınmıştır. Çalışma kapsamında; kaynakları kısıtlı bir edge ortamında spektral analiz ve derin öğrenmeyi entegre eden, gerçek zamanlı izleme ve anomali tespiti yapan bir mimari gerçekleştirilmiştir. Geliştirilen model %99.28 accuracy sağlamış olup sistemin endüstriyel uygulamalarda da kullanılabilirliğini doğrulamaktadır. Bu araştırma, kestirimci bakım için giderek büyüyen Edge AI alanına katkıda bulunarak düşük güçlü mikrodenetleyiciler üzerinde gelişmiş yapay zeka modellerinin etkinliğini ortaya koymaktadır

1 Giriş

Mühendisliğin temel amaçlarından biri; üreticiler arasında çevreye duyarlı ve sürdürülebilir uygulamaları teşvik ederek ürünlerin, süreçlerin ve hizmetlerin tüm yaşam döngüleri boyunca performanslarını ve sürdürülebilirliklerini optimize etmektir[1]. Bu yaklaşım; ekipmanların ömür beklentisini uzatmayı, sık parça veya donanım değişim ihtiyacını azaltmayı, böylece imalat süreçleriyle ilişkili çevresel atıkları en aza indirmeyi beraberinde getirmektedir. Söz konusu sürdürülebilirliği ve yüksek verimliliği sağlamak adına sistematik bir bakım stratejisi zorunludur. Günümüzde endüstride uygulanan üç temel bakım türü bulunmaktadır:

1. Düzeltici Bakım: Ekipmanda bir hata veya arıza oluşuktan sonra gerçekleştirilen, acil onarım veya parça değişimini kapsayan bakım türüdür.
2. Önleyici Bakım: Arızalardan kaynaklanan plansız duruş sürelerini azaltmak amacıyla belirli periyotlarla (aylık/yıllık) planlı ve programlı olarak uygulanan geleneksel bir yöntemdir.
3. Kestirimci Bakım: Ekipman durumuna ilişkin parametreleri sürekli takip eden izleme mekanizmalarına dayanan; bu parametrelerdeki değişimleri analiz ederek olası anomalileri ekipman henüz bozulmadan teşhis eden veya tahmin eden ileri düzey bir yaklaşımdır. Bu sayede bakım masrafları en aza indirilirken, ekipman performansı maksimum düzeyde tutulur ve operasyonel yaşam döngüsü uzatılmış olur.

Ancak kestirimci bakım, operasyonel süreçleri ve hata dinamiklerini derinlemesine anlamayı gerektirdiğinden diğer iki bakım türüne kıyasla uygulanması en zor yöntemdir. Çünkü arıza henüz fiziksel olarak meydana gelmeden, sistemin periyodik düzenini bozan mikro düzeydeki sapmaların (anomalilerin) fark edilmesi gerekir. Bu durum, veri toplama ve işleme süreçlerinde son derece hassas ve "milimetrik" bir ölçüm düzenine olan ihtiyacı doğurur. Diğer bir ifadeyle; sistemin kendi kendine yetebilmesi, bir anomali fark ettiğinde bu durum kalıcı bir arızaya dönüşmeden veriyi yerelde işleyebilmesi ve gerekli aksiyonları alarak yetkili personele anlık bilgi aktarabilmesi gerekmektedir.

Kestirimci bakım süreçlerinde bu karmaşık örüntüleri yakalayabilmek adına yapay zeka (ML/DL) algoritmalarına ihtiyaç duyulmaktadır. Bu algoritmaların koşturulabileceği ve mimari olarak entegre edilebileceği üç farklı bilişim katmanı mevcuttur:

1. Cloud Computing: Özellikle karmaşık işlemler ve yüksek miktarda depolama kapasitesi gerektiren, veri iletim gecikmesine (latency) karşı yüksek hassasiyeti bulunmayan IoT uygulamaları için merkezi ve ölçeklenebilir bir ortam sağlar.[2]
2. Fog Computing: Coğrafi konum olarak IoT cihazlarına bulut sunucularından daha yakın olan sis bilişim düğümleri, merkezi bulut sistemlerine kıyasla veriyi daha yerelde işleyerek daha hızlı yanıt süreleri sunabilmektedir.[2]
3. Edge Computing: Bilgi işlem gücünü doğrudan verinin üretildiği IoT cihazlarının üzerine yerleştirir. Buradaki temel fikir; tüm verileri bir sis bilgi işlem düğümüne veya bulut sunucusuna iletme zorunluluğunu ortadan kaldırarak analizi doğrudan uç cihaz üzerinde gerçekleştirmektir. Cihaz seviyesinde yapılan bu işleme süreci; veri gizliliğini ve güvenliğini artırırken, ağ üzerinden uzak sunuculara gönderilen gereksiz veri trafiğini ve bant genişliği tüketimini de ciddi oranda azaltır.[2]

Bu doğrultuda, projemizde cloud computing'in getirdiği overhead'leri (yüksek gecikme ve ağ bağımlılığı) ortadan kaldırmak için bir edge computing ve TinyML mimarisi kullanılmıştır. Projede kaynakları kısıtlı bir mikrodenetleyici üzerinde spektral analiz yöntemleri ile derin öğrenme modelleri entegre edilerek, bir motor/fan düzeneğinde oluşan anomali'leri milisaniyeler içinde tespit edebilen bir gömülü sistem oluşturduk.

2 Projenin Amacı ve Kapsamı

Bu projenin temel amacı, fan/motor gibi döner sistemlerde oluşabilecek arızaları erken aşamada tespit ederek kestirimci bakım yaklaşımı sağlamaktır. Döner sistemlerde meydana gelen titreşim değişimleri, sistemde oluşabilecek mekanik dengesizliklerin veya arıza başlangıçlarının göstergesi olabilir. Bu nedenle projede, fan titreşim verileri kullanılarak sistemin çalışma durumunun sınıflandırılması hedeflenmiştir.

Bu çalışma endüstriyel bir motor üzerinde değil, laboratuvar ortamında hazırlanan bir fan düzeneği üzerinde gerçekleştirilmiştir. Fan üzerinde normal çalışma, fan kapalı durumu ve anomali durumu oluşturularak veri toplanmıştır. Titreşim verileri MPU6050 ivmeölçer sensörü ile X, Y ve Z eksenlerinden alınmıştır. Elde edilen ivme verileri ESP32 mikrodenetleyicisi üzerinde işlenmiş; filtreleme, vibration score ve RMS hesaplamaları yapılmıştır.

Toplanan veriler Edge Impulse platformuna aktarılmış ve burada fan_off, normal ve anomaly sınıflarını ayırt edebilen bir TinyML sınıflandırma modeli oluşturulmuştur. Eğitilen model daha sonra ESP32 üzerine gömülerek sistemin uç cihaz üzerinde gerçek zamanlı tahmin yapabilmesi sağlanmıştır. Böylece verilerin sürekli olarak uzak bir sunucuya gönderilmesine gerek kalmadan, sınıflandırma işlemi doğrudan mikrodenetleyici üzerinde gerçekleştirilmiştir.

Sistemin uzaktan izlenebilmesi için ESP32 tarafından elde edilen Tahmin sonuçları MQTT protokolü kullanılarak HiveMQ Cloud üzerinden Node-RED dashboard'una aktarılmıştır. Dashboard üzerinde motor durumu, güven oranı, anlık titreşim değeri, titreşim grafiği ve toplam anomali sayısı görüntülenmiştir. Ayrıca RGB LED ile yerel uyarı sistemi eklenmiş, Node-RED üzerinden e-posta bildirimi gönderilebilecek bir uyarı mekanizması da tasarlanmıştır. Bu kapsamda proje, fan titreşimlerinin ölçülmesi, yapay zekâ modeli ile sınıflandırılması ve sonuçların hem yerel hem de uzaktan izlenmesi süreçlerini içermektedir.

3 Kullanılan Donanım ve Yazılımlar

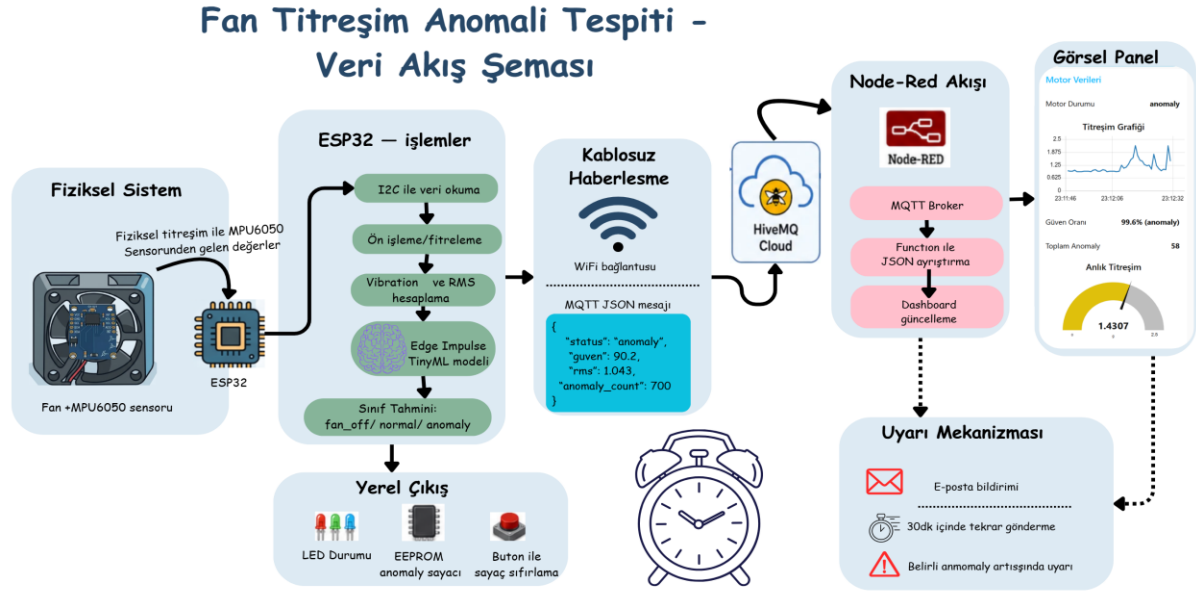
3.1 Donanım Tablosu

Donanım	Görevi
ESP32	Sensör verilerinin okunması, okunan bu verilerin TinyML modeli ile edge sistem üstünden işlem yapıp, Wi-Fi kullanarak MQTT ile bilgileri Node-RED'e gönderme.
Fan	Sistemimizde titreşim oluşturmak için kullanılan elektronik cihaz.
MPU6050	Fan'dan çıkan titreşimleri ve x/y/z eksenleri ivmelerini ölçmek için.
RGB LED / LED'ler	Sistemimizde devre üzerinde fiziksel olarak durum göstermek için (fan_off, anomaly ve normal durumları)
Buton	Sistemde şu ana kadar oluşan bütün anomali sayısını EEPROM'da sıfıra indirmek için.
Breadboard ve jumper kablolar	Devredeki cihazları birbirine bağlamak ve derli toplu gözükmesini sağlamak için kullanıldılar.

3.2 Yazılım Tablosu

Yazılım / Platform	Görevi
Arduino IDE	ESP32'nin programlanması için kullanılmıştır.
Edge Impulse	Sistemde kullanılan TinyML modelinin eğitilmesi ve Arduino IDE'ye uygun kütüphane formatına dönüştürülmesi için kullanılmıştır.
HiveMQ Cloud	MQTT tabanlı veri iletişimi için broker hizmeti sağlamıştır.
Node-RED	Dashboard oluşturma ve veri görselleştirmek için kullanılmıştır.
Python	ESP32'den gelen sensör verilerini TinyML modeline uygun veri seti formatına dönüştürmek için kullanılmıştır.
Gmail SMTP	Tespit edilen anomalilerin e-posta ile bildirilmesi için kullanılmıştır.

4 Sistem Mimarisi



Fiziksel Sistem: Fanda oluşan titreşimlerin x, y, z eksenlerindeki hareketlilikleri okunup ESP32'ye gönderilmektedir.

ESP32 İşlemleri: Veriler alınıp ön işleme ve filtreleme işlemleri yapılmakta, sonrasında Vibration ve RMS hesaplamaları gerçekleştirilmektedir. Daha sonra bu bilgilerin hepsi edge sistemin içindeki TinyML modeline verilip sınıf tahmini yapılmakta ve oradan çıkan sonuca göre yerelde RGB LED'in rengi değiştirilmektedir.

Kablosuz Haberleşme: ESP32'den gelen sonuç, HiveMQ cloud broker aracılığıyla Node-RED ortamına taşınmaktadır.

Node-RED Akışı: JSON formatında gelen bilgi parçalanıp dashboard anlık olarak güncellenmekte; oluşan anomaly sayısı belirli bir sınırı geçtiğinde e-mail aracılığı ile bir uyarı merkezi kurgulanmaktadır.

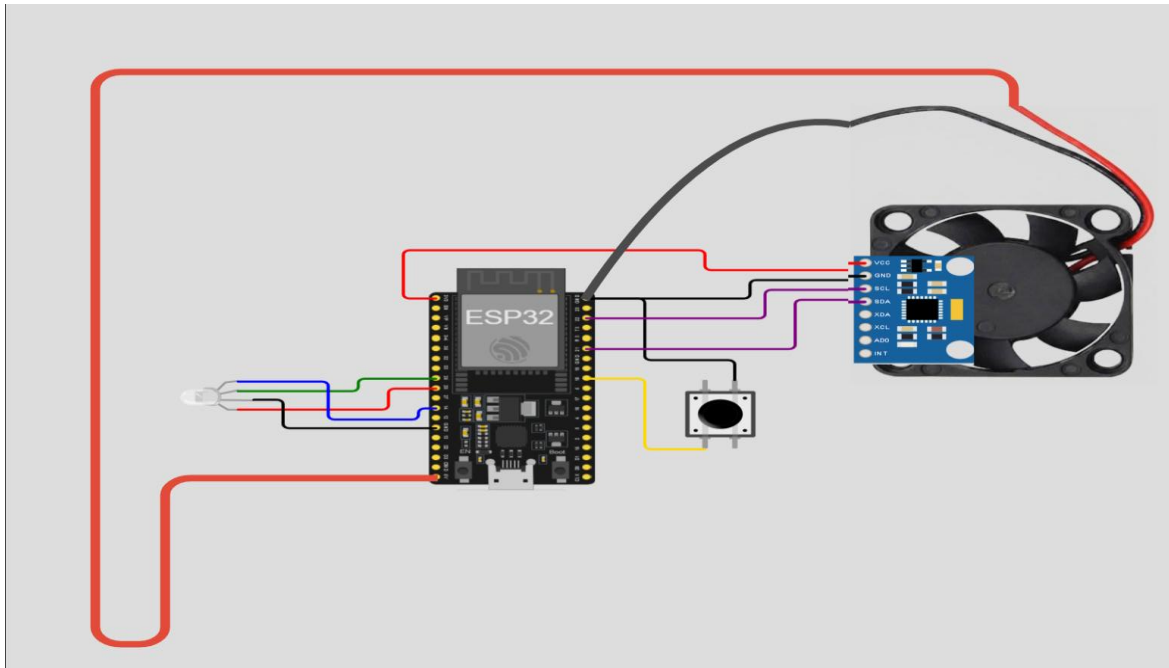
Yerel Çıkış: Oluşan anomaly sayısı EEPROM'da tutulup belirli bir sayıyı geçtiğinde uyarı mekanizmasındaki işlemler gerçekleşmektedir. İşlemler bittiğinde kullanıcı tarafından butona basılıp anomaly sayacı sıfırlanmaktadır.

5 Devre Bağlantısı ve Devre Şeması

5.1 Devre Bağlantısı Tablosu

Modül / Bileşen	Modül Üzerindeki Pin	ESP32 Karşılığı (GPIO)	Sinyal Tipi
MPU6050 İvmeölçer	VCC	3V3	Güç
	GND	GND	Toprak
	SDA	GPIO 21	Dijital (I2C)
	SCL	GPIO 22	Dijital (I2C)
Fan	Kırmızı Kablo (+)	VIN (5V)	Güç
	Siyah Kablo (-)	GND	Toprak
RGB Durum LED'i	Kırmızı Anot (R)	GPIO 26	Çıkış (PWM)
	Yeşil Anot (G)	GPIO 25	Çıkış (PWM)
	GPIO 14	GPIO 14	Çıkış (PWM)
	Ortak Katot (-)	GND	Toprak
Butonu	Birinci Uç	GPIO 18	Giriş (Interrupt)
	İkinci Uç	GND	Toprak

5.2 Devre Şeması



6 Veri Toplama Süreci

Veriler üç farklı durumda toplanmıştır: *fan_off*, *normal* ve *anomaly*. *fan_off* durumu için motorun ESP32'ye giden voltaj bağlantısı çıkarılmış ve sistem 3 dakika boyunca çalıştırılmıştır. Bu süreçte sensör verileri seri monitöre yazdırılmış, ardından DatasetOlusturma.py (Şekil A'daki kod) çalıştırılarak elde edilen veriler CSV formatında kaydedilmiştir (Şekil B'de gösterildiği gibi). Daha sonra normal durum için motor, ESP32'nin 5V pinine bağlanmış ve aynı veri toplama süreci uygulanarak veriler kaydedilmiştir. Son olarak anomaly verisi için fanın pervanesine bir cisim bağlanarak yapay bir anomaly oluşturulmuş ve ilgili veriler toplanmıştır. Toplanan tüm veriler, daha sonra TinyML modelinin dataset'i olarak kullanılmıştır.

```

7 import serial
8 import time
9
10 PORT = "COM10"
11 BAUD = 115200
12 DOSYA_ADI = "anomaly.csv"
13 KAYIT_SURESI = 60 # saniye, 3 dakika
14
15 ser = serial.Serial(PORT, BAUD, timeout=1)
16 time.sleep(2)
17
18 baslangic = time.time()
19
20 with open(DOSYA_ADI, "w", encoding="utf-8") as f:
21     print("Kayit basladi...")
22
23     while time.time() - baslangic < KAYIT_SURESI:
24         satir = ser.readline().decode("utf-8", errors="ignore").strip()
25
26         if satir:
27             print(satir)
28             f.write(satir + "\n")
29
30 ser.close()
31 print("Kayit bitti:", DOSYA_ADI)

```

```

1 timestamp,ax,ay,az,vibration,rms
2 1437,-2.0000,1.9999,0.8169,0.8832,0.8832
3 1457,1.3274,-2.0000,0.8347,0.9869,0.4593
4 1477,0.8105,1.9999,1.1726,0.7354,0.8339
5 1497,-2.0000,-1.3960,0.5764,0.8729,0.7703
6 1517,1.9999,-1.6692,1.0227,0.8806,0.9775
7 1537,-2.0000,1.9999,1.0603,1.0355,0.9510
8 1557,0.4658,-2.0000,0.7314,0.7115,0.9368
9 1577,1.4878,1.9999,1.2437,0.8929,1.0401
10 1597,-2.0000,-0.1221,0.7537,0.7175,0.9648
11 1617,1.9999,-2.0000,0.9084,0.9587,1.0745
12 1637,-1.6235,1.9999,1.1563,0.9563,1.0326
13 1657,-0.3975,-2.0000,0.6978,0.6841,1.0236
14 1677,1.9999,1.9692,1.1852,0.9943,1.0754
15 1697,-2.0000,1.0801,0.8584,0.7574,1.1144
16 1717,1.9999,-2.0000,0.8098,1.0311,0.9927
17 1737,-0.9495,1.9999,1.0503,0.7789,1.0370
18 1757,-1.1543,-2.0000,0.7314,0.8096,0.9918
19 1777,1.9999,1.0249,0.9836,0.8118,0.9779
20 1797,-2.0000,1.9999,0.8083,0.9057,1.1625
21 1817,1.9999,-2.0000,0.7876,1.0694,0.9293
22 1837,-0.2429,1.9999,0.9531,0.6638,1.0328
23 1857,-1.8152,-2.0000,0.6614,0.9626,0.9510
24 1877,1.3662,1.9999,1.2480,0.8708,1.0445
25 1897,-2.0000,-0.4207,0.7014,0.6872,1.0308
26 1917,1.9999,-2.0000,0.9343,0.9989,1.0379
27 1937,-1.7861,1.9999,1.1519,0.9625,1.0712
28 1957,-0.1101,-2.0000,0.7122,0.6885,1.0262
29 1977,1.9172,1.9999,1.1838,0.9858,1.0921

```

Şekil-A DatasetOlusturma.py

Şekil-B anomaly.csv

ESP32'nin hangi portta olduğu yazıldıktan sonra, verileri düzgün yakalamak için baud hızı yazılmıştır. Daha sonra gelen verilerin hangi duruma ait olduğuna göre bir CSV dosyası oluşturulmuştur (ekran görüntüsünde anomaly verileri geldiği için anomaly dosyası oluşturulmuştur). Seri haberleşme başlatılıp timer başlatıldıktan sonra gelen veriler kaydedilmiştir. Veriler Şekil-B'de olduğu gibi kaydedilmektedir:

Timestamp: Gelen verilerin hangi zamanda geldiğini göstermektedir.

ax: x eksenindeki ivme değeri.

ay: y eksenindeki ivme değeri.

az: z eksenindeki ivme değeri.

vibration: Titreşim miktarı.

rms: Kök Ortalama Kare değeri.

```

float vibration = mySqrt(farkX*farkX + farkY*farkY + farkZ*farkZ);
float rms       = mySqrt(filtAx*filtAx + filtAy*filtAy + filtAz*filtAz);

```

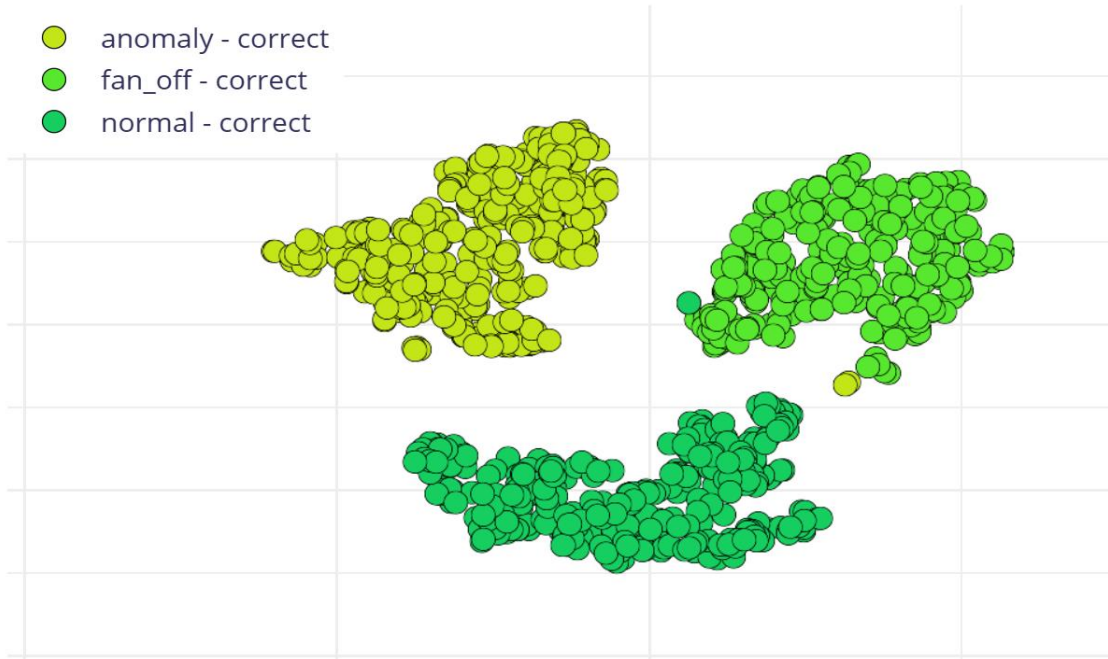
Vibration ve RMS yukarıda gösterildiği gibi hesaplanmaktadır. Projemizde RMS'nin kullanılmasının amacı kısaca şöyle özetlenebilmektedir: Literatürde, titreşim izleme tabanlı anomali tespiti uygulamalarında RMS (Root Mean Square) kullanımı özellikle tavsiye edilmektedir; zira RMS yöntemi ani sinyal sıçramalarına ve çevresel gürültülere karşı daha az duyarlı olup, sinyalin toplam kinetik enerjisini çok daha kararlı ve güvenilir bir şekilde temsil etmektedir.[1]

7 Edge Impulse Model Eğitimi

Ham (raw) ivmeölçer verileri gürültülü, hatalı ve bağlantılı olmayan verilerden oluşabilmektedir; bu yüzden filtreleme ve normalizasyon işlemleriyle verilerin daha güvenilir ve özellik çıkarımı (feature extraction) için daha uygun hale getirilmesi sağlanmıştır. Edge Impulse platformunda varsayılan (default) olarak normalizasyon özelliği bulunmadığı için bu işlemin veri yüklenmeden önce yapılması gerekmektedir. [1]

Verilerin doğası gereği, segmentasyon işlemi yapılarak verinin daha küçük segmentlere veya pencerelere (windows) parçalanması gerekebilmektedir. Bu sayede, veriden spesifik olarak daha düzgün ve yönetilebilir bir özellik çıkarımı gerçekleştirilebilmektedir. Pencere boyutu (window size) ve örtüşme miktarı (overlap), uygulamaya ve verinin oluşan durum hakkındaki bilgi vericiliğine bağlıdır.[1] Bu projede pencere boyutu (window size) 1000 ms, pencere artış miktarı (window increase) ise 200 ms olarak ayarlanmıştır. Modelde kullanılan bütün katmanların sayısı, çıkışta ve girişteki nöron sayısı gibi kullanılan tüm özellikler ve bu ayarlamalardan sonra eğitim aşamasında oluşan sınıf ayrımları Şekil-C'de açık olarak görülmektedir.

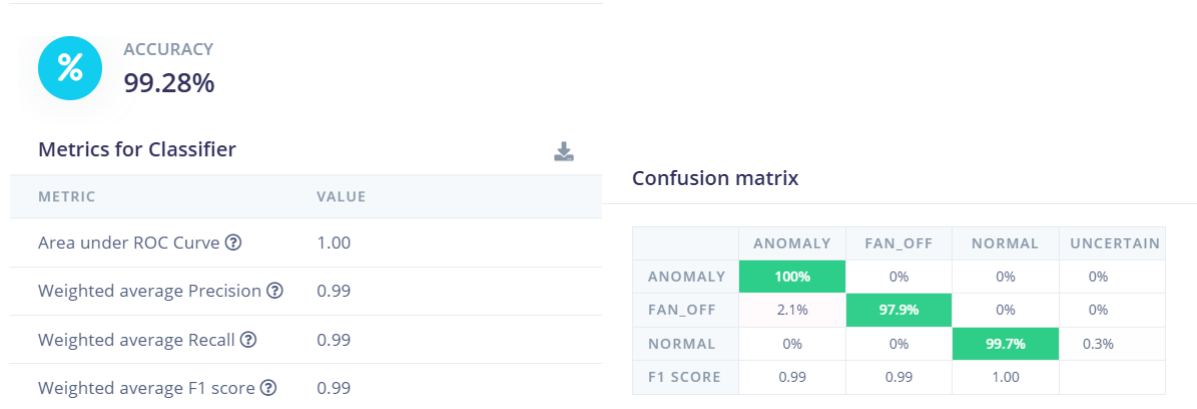
Şekil-C Neural Network settings



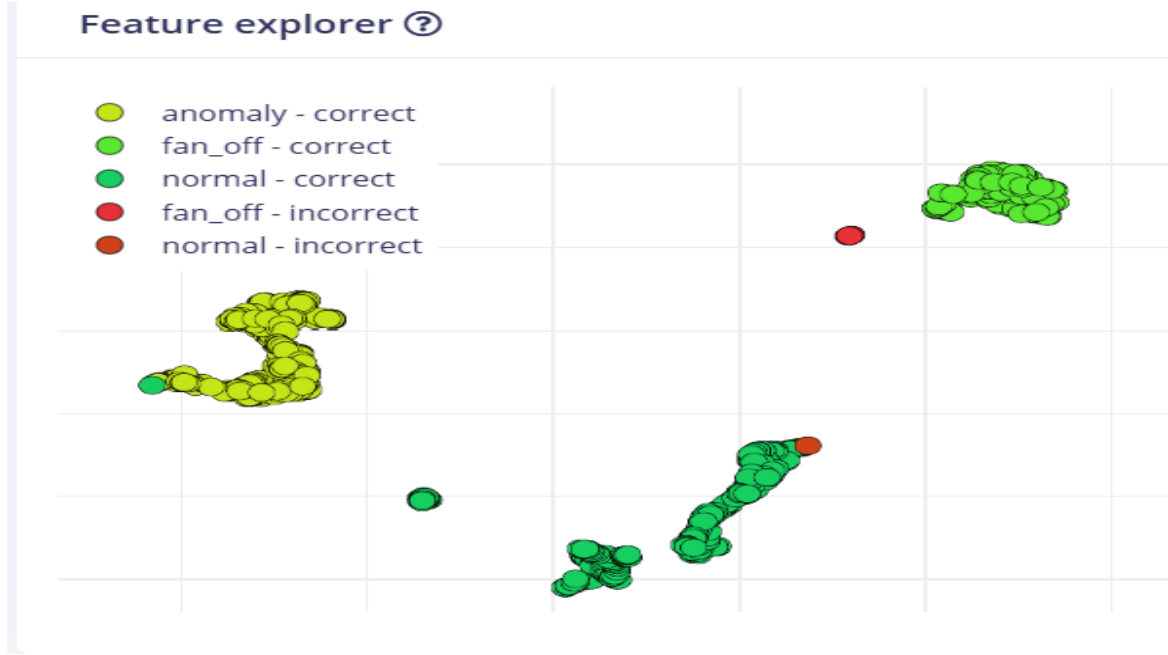
Şekil-C Data explorer (classified samples)

7.1 Model Sonuçları

Model test sonucunda elde edilen doğruluk (accuracy) değeri %99.28 olarak gerçekleştirilmiş olup bu değer, modelimizin çok iyi bir şekilde eğitildiği ve neredeyse hatasız çalıştığı anlamına gelmektedir. Modelin test süreci sonunda elde edilen doğruluk (accuracy), karmaşıklık matrisi (confusion matrix) ve özellik gezgini (feature explorer) grafikleri ile tabloları altta gösterilmektedir.



Şekil-D: Test sonunda oluşan doğruluk (accuracy), karmaşıklık matrisi (confusion matrix)



Şekil-D: Özellik gezgini (feature explorer) görüntüleri.

8 Bu Projede Neden TinyML Modeli Kullanıldı?

TinyML modelinin hem düşük güç tüketimine hem 25 ms gibi hızlı bir yanıt (response) süresine hem de doğrusal (linear) bir karmaşıklığa sahip olması nedeniyle; Random Forest ve SVM modellerine kıyasla katbekat daha kullanışlı olduğu gösterilmektedir.[1]

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 score (%)	Inference time (ms)	Computational complexity
TinyML	96.5	96.19	97.11	96.64	25	Low ($O(N)$)
Random Forest	93.5	91.0	90.4	90.7	60	Medium ($O(n \cdot \log(n))$)
SVM (RBF Kernel)	89.8	88.1	87.5	87.8	78	High ($O(n^2)$)

Şekil-F: Önerilen kestirimci bakım sistemi için ve Random Forest ve SVM modellerinin detayli karsilastirma tablosu[1]

9 TinyML'in ESP32'ye Bağlanması

Oluşturulan model Arduino kütüphanesi olarak indirilip Arduino IDE'sine yüklendikten sonra, kullanılmaya hazır hale gelmesi için alttaki fonksiyonlar yazılmıştır:

```
float features[EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE]; //oto aliyor kendisine en uygununu
```

Böyle yapılarak bir özellik (feature) listesi oluşturulmuştur. Daha sonra alttaki kod yazılmıştır:

```

signal_t signal;
signal.total_length = EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE;
signal.get_data = &getFeatureData;

ei_impulse_result_t result = {0};
EI_IMPULSE_ERROR res = run_classifier(&signal, &result, false);
if (res != EI_IMPULSE_OK) {
    Serial.print("Classifier hatasi: ");
    Serial.println(res);
    return;
}

```

Bu kod parçasında ise şunlar gerçekleşmektedir: İlk başta bir sinyal verisini tutan bir değişken tanımlanmış, ardından bu değişkenin toplam uzunluğu *EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE* olarak atanmıştır. (*EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE* değeri, pencere boyutu [window size] ile eksen/özellik sayısının (5) çarpılmasıyla hesaplanmaktadır.)

Daha sonra, *ei_impulse_result_t* *result* değişkeni ile yapay zekanın yapacağı tahmin sonuçlarını içine kaydetmesi için bir yapı tanımlanmıştır. Sonrasında *run_classifier* fonksiyonuna yazılan tüm değişkenler verilmiş; tahmin gerçekleştirilmişse tahminin söylenmesi, hata çıkacaksa hata mesajının ekrana yazdırılması sağlanmıştır.

10 Karşılaşılan Problemler ve Çözümler

Projede karşılaşılan ilk problem; DC motorun yaydığı yüksek elektromanyetik dalgalar (EMI) nedeniyle MPU6050 ivmeölçer sensörünün veri okuyamaması olmuştur. Bu sorunun çözümü için ilk olarak I2C haberleşme hatlarına (SDA ve SCL kanallarına) pull-up dirençleri bağlanarak hatlara sabit bir voltaj verilmiş ve sinyal kararlılığı bir nebze olsun artırılmıştır. Ancak bu müdahale, veri kesintilerini tamamen engelleyememiştir.

Bir sonraki adım olarak parazitleri sönmölemek amacıyla MPU6050'nin sinyal kabloları saç örgüsü şeklinde birbirine dolanmıştır. Bu yöntem de hatayı tamamen ortadan kaldıramadığı için sisteme dekaplaj kondansatörleri eklenmesi veya DC motor yerine titreşim oluşturabilecek daha kararlı başka bir düzeneğin kullanılması gerekliliği doğmuştur. Bu doğrultuda, DC motor yerine fırçasız (brushless) bir fan kullanılmasına karar verilmiş ve elektromanyetik parazit kaynaklı veri okuma hatalarının tamamen önüne geçilmiştir.

Proje kapsamında karşılaşılan bir diğer problem ise; peş peşe anomali durumlarının tespit edilmesi halinde sistemin art arda çok sayıda e-posta göndermesi ve bu sebeple e-posta sunucusu tarafından engellenmesi (bloklanması) olmuştur. Bu sorunun çözümü için anomali sıklığına dayalı bir sınır (eşik) değeri belirlenmiş ve uyarı mekanizmasının yalnızca bu eşik değeri aşıldığında e-posta bildiriminde bulunması sağlanmıştır.

11 Sonuç ve Değerlendirme

Bu proje ile endüstriyel ve kaynak kısıtlı ortamlarda çalışmak üzere özel olarak tasarlanmış, TinyML destekli bir ESP32 gömülü sistemi aracılığı ile fan düzeneklerindeki arızaların tespitine yönelik bir çalışma ortaya konulmuştur. Sistem; bir uç cihaz (edge device) üzerinde spektral analiz yöntemlerini ve derin öğrenme modellerini entegre ederek gerçek zamanlı anomali tespit sistemi ve yerel geri bildirim üretimi sağlamıştır. Böylece bulut bilişimin (cloud computing) getirdiği yüksek ek yükler (yüksek gecikme ve ağ bağımlılığı) gibi zorluklar etkili bir şekilde çözülmüştür. Çalışmada, gelişmiş makine öğrenmesi modelleri kısıtlı kaynaklara sahip mikrodenetleyici üzerinde başarıyla koşturulmuştur; bu da beklenmedik arızaları ve bakım maliyetlerini en aza indirerek kestirimci bakıma önemli ölçüde katkıda bulunmaktadır.

Gerçekleştirilen eğitim ve test süreci sonucunda modelimizin son derece etkili olduğu kanıtlanmış ve modelimiz %99.28 gibi yüksek bir doğruluk (accuracy) değerine ulaşmıştır. Sistemde gerçek zamanlı veri iletimi için HiveMQ cloud broker altyapısı ve Node-RED akışı kullanılmıştır. Anlık durum takibi için bir Node-RED dashboard ekranı ve belirlenen anomali sınırı geçildiğinde tetiklenen bir e-mail uyarı merkezi kurgulanmıştır.

Ayrıca, oluşan anomali sayılarının EEPROM üzerinde kalıcı olarak tutulması ve belirli bir sayı aşıldığında uyarı mekanizmasının işletilmesi sağlanmıştır. İşlemler bittiğinde kullanıcının fiziksel bir buton vasıtasıyla anomali sayacını yerelde sıfırlayabilmesi ise sisteme pratik bir kontrol yeteneği kazandırmıştır.

Geleneksel yöntemler ve mevcut baz modeller ile karşılaştırıldığında, TinyML modelimiz hem düşük güç tüketimi hem 25 ms gibi hızlı bir yanıt (response) süresi hem de doğrusal (linear) bir karmaşıklığa sahip olması nedeniyle Random Forest ve SVM modellerine kıyasla katbekat daha üstün ve kullanışlı bir performans elde etmektedir. Bu durum, geliştirilen sistemi çeşitli fırçasız motor ve fan düzeneklerinde ölçeklenebilir uygulamalar için tamamen uygun hale getirmektedir. Sonuç olarak bu araştırma; kestirimci bakım sistemlerinin artık merkezi bulut altyapılarına yoğun bir şekilde bağımlı kalmadan, veri kaynaklarına en yakın noktada (uçta) ve milisaniyeler mertebesinde kendi kendine yeterek çalışabileceğini başarıyla ortaya koymaktadır.[1]

Kaynakça

- [1] S. Arciniegas, D. Rivero, J. Piñan, E. Diaz, and F. Rivas, “IoT device for detecting abnormal vibrations in motors using TinyML,” Discover Internet of Things, vol. 5, article 41, 2025, doi: 10.1007/s43926-025-00142-4.
- [2] Alperen Eroglu IoT Ders Notları
- [3] Shahad Alharthi *, MuhammadRashid andMalakAljabr , “TinyML in Industrial IoT: A Systematic Review of Applications, System Components, and Methodologies”
- [4] Saima Zafar , Ghosia Miraj, Rajaa Baloch , Danish Murtaza, Khadija Arshad, “An IoT Based Real-Time Environmental Monitoring System Using Arduino and Cloud Service”
- [5] Massimo Banzi – Arduino, “Tiny machine learning with Arduino”